

Video 1.3: The Shared Kernel Problem

Duration: 7:00

Format: PPT (25%) / Terminal (40%) / Code Editor (20%) / Browser (15%)

Resolution: 1920x1080 @ 30fps

Prerequisites: Video 1.1, 1.2

SCRIPT BEGIN

[0:00-0:40] — Opening: Why the Kernel Matters (PPT Slide)

SLIDE: Title: “The Shared Kernel Problem” — Central kernel icon with arrows pointing to it from 6 container icons, each labeled “Container 1” through “Container 6”

NARRATION:

“In the previous lesson, we established that containers share the host kernel. In this lesson, we’re going to explore exactly why that’s a problem. The Linux kernel exposes over 400 system calls. Every single one of those system calls is available to every container on the host — unless you explicitly restrict them. The kernel is the largest shared attack surface in any container deployment, and one vulnerability in one system call can compromise every container on the machine.”

[0:40-1:40] — System Call Exposure (Terminal)

SCREEN: Terminal at 150% zoom

NARRATION:

“Let me show you the scale of this surface.”

COMMAND + EXPLANATION:

```
# Count available system calls on the host kernel
$ ausyscall --dump | wc -l
```

“On this kernel, there are over 350 system calls. Every one of these is callable from inside a container by default.”

```
# Show some security-relevant system calls
$ ausyscall --dump | grep -E "mount|ptrace|reboot|kexec|bpf|userfault"
```

“Look at some of these — mount, ptrace, reboot, kexec_load, bpf, userfaultfd. These are powerful system calls. `mount` can attach filesystems. `ptrace` can trace and control other processes. `kexec_load` can load a new kernel. `bpf` can inject code into the kernel. Not all of these are available to containers by default — Docker does apply a default Seccomp profile — but the point is that the kernel surface is vast, and any unfiltered call is a potential attack vector.”

```
# Check Docker's default seccomp profile – how many calls are blocked?
$ docker run --rm alpine cat /proc/1/status | grep Seccomp
Seccomp:      2
Seccomp_filters: 1
```

“Seccomp mode 2 means the container is running with a Seccomp filter. Docker’s default profile blocks about 44 system calls. That means roughly 300+ calls are still permitted. We’ll deep-dive into Seccomp in Module 3, but the takeaway here is: even with defaults, the attack surface is huge.”

[1:40-3:00] — Kernel Vulnerability Impact (Browser + PPT)

NARRATION:

“Let’s see what happens when a kernel vulnerability is discovered.”

BROWSER: Navigate to <https://nvd.nist.gov/vuln/detail/CVE-2022-0185>

NARRATION:

“CVE-2022-0185 — a heap-based buffer overflow in the Linux kernel’s filesystem context handling. Published January 2022. CVSS score 8.4. This vulnerability allowed an unprivileged user — or a container process — to escalate privileges to root on the host. Let me read the key part: ‘a heap-based buffer overflow flaw was found in the legacy_parse_param function in the Filesystem Context functionality of the Linux kernel.’ All a container needed was the CAP_SYS_ADMIN capability, which privileged containers have by default.”

SWITCH TO PPT SLIDE:

SLIDE: Title: “CVE-2022-0185 — Impact Analysis”

- **Affected:** Linux kernel 5.1 through 5.16.2
- **Vector:** From inside a container with CAP_SYS_ADMIN
- **Impact:** Container escape → host root access
- **Scope:** EVERY container on the affected host is compromised
- **Fix:** Kernel update required — not a container image update
- **Timeline:** Disclosed Jan 18, 2022 → Exploit public Jan 25, 2022 (7 days)

NARRATION:

“Here’s the critical lesson. When a kernel vulnerability is found, you can’t fix it by updating your container images. You can’t fix it by restarting your containers. You must patch the host kernel and reboot. And until you do, every single container on that host is potentially exploitable. The shared kernel means a single vulnerability has a blast radius of the entire host. In a VM model, each VM has its own kernel — a vulnerability in the guest kernel doesn’t affect the host. That’s the fundamental difference.”

[3:00-4:30] — Demonstrating Cross-Container Visibility (Terminal)

NARRATION:

“Let me demonstrate another consequence of kernel sharing — cross-container visibility.”

COMMAND + EXPLANATION:

```
# Start two "isolated" containers
$ docker run -d --name victim alpine sleep 3600
$ docker run -d --name attacker alpine sleep 3600
```

“I’ve started two containers — victim and attacker. They should be completely isolated from each other, right?”

```
# From the HOST, find both container PIDs
$ VICTIM_PID=$(docker inspect --format '{{.State.Pid}}' victim)
$ ATTACKER_PID=$(docker inspect --format '{{.State.Pid}}' attacker)
$ echo "Victim: $VICTIM_PID, Attacker: $ATTACKER_PID"
```

“From the host, both containers are just processes with PIDs.”

```
# If the attacker escapes to the host (e.g., via kernel exploit),
# they can see everything
# Simulating host-level access:
$ ls /proc/$VICTIM_PID/environ
$ cat /proc/$VICTIM_PID/environ | tr '\0' '\n'
```

“With host-level access, the attacker can read the victim container’s environment variables — where credentials are often stored. They can read its command line, its open file descriptors, its network connections. The PID namespace hides these from inside the container, but from the host, everything is visible. This is why a container escape is catastrophic — it’s not just one container that’s compromised.”

```
# Show the victim's network connections from host
$ nsenter --target $VICTIM_PID --net ss -tlnp
```

“Using `nsenter`, I can enter the victim’s network namespace and see its open ports and connections. This is the same tool Docker uses internally — and it’s what an attacker uses after escaping.”

```
# Clean up
$ docker rm -f victim attacker
```

[4:30-5:30] — Kernel Module Risk (Code Editor + Terminal)

SCREEN: Code editor at 150% zoom

NARRATION:

“There’s another dimension to the shared kernel problem — kernel modules.”

CODE EDITOR — Showing a conceptual attack flow:

SHARED KERNEL – MODULE ATTACK FLOW

=====

1. Attacker gains root inside a privileged container
2. Attacker loads a malicious kernel module:
\$ insmod /tmp/rootkit.ko
3. Module runs in kernel space – NO isolation applies
4. Module can:
 - Intercept system calls of ALL containers
 - Hide processes from ALL containers
 - Capture network traffic of ALL containers
 - Modify file I/O of ALL containers
 - Install persistent backdoor on the HOST
5. Module survives container restart
6. Module is invisible to container-level monitoring

NARRATION:

“If an attacker gains root inside a privileged container, they can load kernel modules. And a kernel module runs in kernel ring 0 — it’s completely outside any container isolation. A malicious kernel module can intercept system calls from every container, capture network traffic, hide processes, and persist across container restarts. This is why privileged containers are so dangerous — and we’ll cover that in depth in Module 2.”

SWITCH TO TERMINAL:

```
# Show that privileged containers can list kernel modules
$ docker run --rm --privileged alpine lsmod | wc -l
```

“A privileged container can list all kernel modules.”

```
# Show that unprivileged containers cannot
$ docker run --rm alpine lsmod 2>&1
lsmod: can't change directory to '/lib/modules/...': No such file or directory
```

“An unprivileged container can’t — but that’s a userspace restriction, not a kernel enforcement. The system call itself might still be available unless Seccomp blocks it.”

[5:30-6:20] — Mitigations Overview (PPT Slide)

SLIDE: Title: “Mitigating the Shared Kernel Risk”

Mitigation	What It Does	Limitation
Seccomp profiles	Restrict available syscalls	Doesn't prevent kernel bugs in allowed calls
AppArmor / SELinux	Mandatory access control	Complex to configure, may break apps
User namespaces	Map container root to unprivileged host user	Not universally supported
Read-only root FS	Prevent filesystem writes	Requires app changes
Kata Containers / gVisor	Add kernel boundary	Performance overhead
Regular kernel patching	Fix known vulnerabilities	Zero-days still a risk
Minimal base images	Reduce userspace attack surface	Doesn't reduce kernel surface

NARRATION:

"There are several mitigation strategies, and we'll cover each one in detail throughout this course. Seccomp profiles restrict which system calls are available — Module 3 is entirely dedicated to this. AppArmor and SELinux provide mandatory access control. User namespaces can remap the container's root user to an unprivileged host user. Read-only filesystems prevent writes. Alternative runtimes like Kata Containers and gVisor add a kernel-level boundary at the cost of performance. And of course, regular kernel patching. No single mitigation is sufficient — defense-in-depth is required. Every layer you add reduces the blast radius of a kernel compromise."

[6:20-7:00] — Key Takeaways and Bridge (PPT Slide)

SLIDE: Title: "The Shared Kernel — Summary"

1. **300+ system calls** are available to containers by default
2. **One kernel vulnerability** can compromise every container on the host
3. **Container escapes** grant access to all co-tenant containers
4. **Kernel modules** bypass all container isolation entirely
5. **Mitigation requires layered defenses** — no single tool is sufficient
6. **Kernel patching cadence** must match your risk tolerance

NARRATION:

"The shared kernel is not a design flaw — it's a design choice that prioritizes performance. But it means your security strategy must account for the reality that containers don't have the hard isolation boundary that VMs provide. You need Seccomp, AppArmor, user namespaces, runtime monitoring, and a disciplined kernel patching cadence — all working together. In the next lesson, we'll shift from architecture to Docker specifically — looking at the Docker daemon, its configuration, and the most common misconfigurations that lead to host compromise."

SLIDE: "Next: Docker Security Basics and Daemon Configuration →"

SCRIPT END

Post-Production Notes

- CVE browser page should be zoomed to 150% for readability — pre-load the page
- For the cross-container demo, use split terminal panes: left for commands, right showing the output
- The kernel module attack flow in the code editor should be revealed line by line with slight delays
- Mitigation table should use color coding: green for effective, yellow for partial, red for limitations
- Add a counter animation on the “300+ system calls” statistic
- Ensure all container images (alpine) are pre-pulled before recording